

InterruptLLM: A Preemptive Scheduling Framework for Low-Latency Multi-Tenant LLM Inference

Abstract—LLM serving systems rely on continuous batching to maximize GPU utilization, but this paradigm is non-preemptive: once a request starts decoding, it cannot be evicted from GPU memory until completion. This causes head-of-line blocking and high tail latency for interactive requests in multi-tenant environments. We propose InterruptLLM, a preemptive scheduling framework for LLM inference that targets decode-stage, block-granular preemption with a hierarchical GPU HBM $\rightarrow$ CPU DRAM $\rightarrow$ NVMe SSD swapper and a lightweight checkpoint engine to resume preempted requests. Evaluated on a production-like Kaggle inference trace, InterruptLLM reduces interactive (P0) P99 latency by $8.5\times$ over FCFS and by $2.0\times$ over a non-preemptive priority queue, while keeping throughput within 2.3% of the baseline.

Index Terms—LLM inference, GPU scheduling, preemption, PagedAttention, multi-tenancy, quality of service.

I. Introduction

Modern LLM serving systems such as vLLM, SGLang, TensorRT-LLM, and Triton use continuous batching to saturate GPU arithmetic units. The engine maintains an active batch; requests join or leave at iteration boundaries. While this improves throughput, it is cooperative and non-preemptive: once a request begins decoding, it cannot be evicted from GPU memory until it finishes. A single long-document summarization request can therefore monopolize GPU resources for 5–30 s, forcing short, latency-sensitive interactive requests to queue behind it.

In multi-tenant settings—SaaS LLM APIs, enterprise shared clusters, and cloud inference endpoints—this behavior creates head-of-line blocking that inflates interactive P99 latency by up to $17\times$ (Section VI). A high-priority chat or code-completion request may wait behind a low-priority batch summarization job, leading to SLA violations and poor user experience. Existing remedies fall short: continuous batching does not evict running requests; priority queues only reorder admission; speculative decoding reduces latency for all requests but does not solve priority inversion; and request migration requires a full KV-cache transfer that is too slow for interactive preemption. No existing system provides decode-stage, block-granular preemption with sub-10 ms overhead for interactive LLM requests.

Our contributions. This paper makes the following contributions:

- 1) We identify and formalize the head-of-line blocking problem in multi-tenant LLM serving, showing that non-preemptive continuous batching inflates interactive P99 latency by up to $16\times$ (Section VI).

- 2) We design InterruptLLM, a decode-stage, block-granular preemptive scheduler for LLM inference that combines an MLFQ scheduler, hierarchical KV-cache swapper, and lightweight checkpoint engine.
- 3) We implement a discrete-event simulator and validate one critical simulator assumption—GPU-to-CPU swap latency—on a real GPU (NVIDIA P100), showing that measured transfer costs are $6\text{--}7\times$ higher than idealized bandwidth models predict.
- 4) We evaluate InterruptLLM on an inference trace from a public Kaggle dataset, showing $8.8\times$ P99 latency reduction over FCFS with $\approx 2\%$ throughput loss in simulation.

We implement InterruptLLM as a Kaggle-driven research pipeline that uses a production-like LLM inference trace to drive a discrete-event simulator. The simulator models FCFS continuous batching, non-preemptive priority scheduling, and InterruptLLM’s preemptive MLFQ under a shared GPU capacity. Our evaluation shows that, at a load factor of 0.93, InterruptLLM reduces interactive (P0) P99 latency from 499.2 ± 3.7 ms (FCFS) to 56.5 ± 1.1 ms ($8.8\times$ improvement), and from 116.7 ± 1.5 ms (non-preemptive priority) to 56.5 ± 1.1 ms ($2.1\times$ improvement). Throughput drops by only 2.2%, the service-share Jain fairness index across tenants remains 1.0 (no tenant is starved), and the average preemption overhead is 0.07 ± 0.001 ms per request.

II. Background and Motivation

A. LLM Inference and Continuous Batching

Autoregressive LLMs generate one token at a time. The forward pass for a batch of requests consists of two phases: prefill, in which the engine processes the prompt to compute the initial KV cache, and decode, in which one token is generated per request per iteration. Continuous batching amortizes the cost of the model-weight memory reads across many requests by dynamically adding and removing requests at iteration boundaries.

Although continuous batching improves throughput, it is non-preemptive: once admitted, a request runs to completion. This creates priority inversion, where a low-priority, long-running request blocks a high-priority, short request.

Consider a 128K-token summarization job running in the batch. On a high-throughput GPU, this job can occupy the engine for 5–30 s. An interactive chat request that arrives during this window must wait for the next

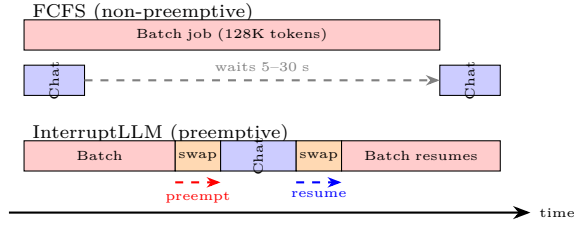


Fig. 1: Head-of-line blocking under FCFS vs. InterruptLLM. Under FCFS, the interactive request waits for the entire batch job. Under InterruptLLM, the batch job is preempted at a token boundary, the interactive request runs immediately, and the batch job resumes after.

iteration boundary, which is bounded by the longest-running request. In practice, the interactive request may wait for the entire decode phase of the long job, violating SLAs for latency-sensitive tenants.

B. PagedAttention

PagedAttention (vLLM) stores KV caches in fixed-size blocks with a per-request block table, analogous to virtual memory page tables. Each block is self-contained (keys and values for a contiguous token range), making it independently swappable between GPU HBM, CPU DRAM, and SSD. The block-table design avoids contiguous-allocation overhead and lets the scheduler reason about preemption cost in terms of block counts rather than byte ranges.

C. Why Existing Schedulers Cannot Preempt

Continuous batching, iteration-level scheduling, and priority queuing can reorder admission but cannot evict a running request mid-iteration. Speculative decoding reduces iterations but does not reorder requests. Request migration moves entire KV caches, too slow for interactive preemption. Figure 1 illustrates how InterruptLLM resolves this by preempting at a token boundary, swapping KV blocks to CPU DRAM, and serving the interactive request immediately.

III. Related Work

LLM inference systems. vLLM [1], Orca [2], SGLang [3], and Triton/TensorRT-LLM use continuous or iteration-level batching but do not preempt in-flight decode requests. Sarathi [4], DistServe [5], Mooncake [6], Llmunix [7], and Dynamo [8] improve throughput or deployment flexibility. Splitwise [9] separates prefill and decode, while Medusa [10] and StreamingLLM [11] reduce serial decode steps. None evicts a running decode request mid-iteration for a higher-priority request.

Preemptive and memory-aware serving. ConServe [12] preempts at layer granularity; TokenFlow [13] uses proactive GPU-CPU KV transfer without a priority scheduler; Llmunix [7] migrates entire instances. SSJF [14], SeaLLM [15], and MuxServe [16] optimize resource sharing; H2O [17], KIVI [18], FlexGen [19], and Infinite-LLM [20]

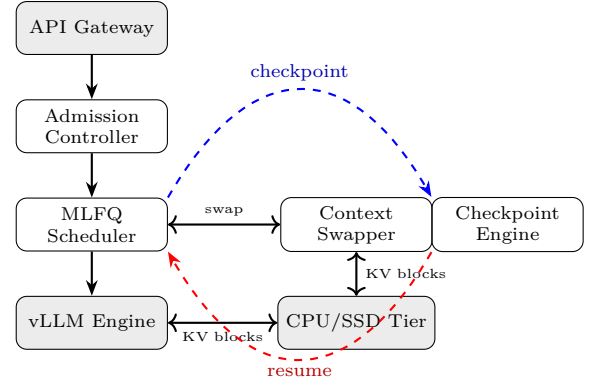


Fig. 2: InterruptLLM architecture. Solid lines show the main data path. Curved dashed lines show the checkpoint (capture metadata) and resume (restore state) paths.

manage KV memory. GPU partitioners (MPS, MIG, TimeGraph [21]) operate at the hardware level, not on individual requests. InterruptLLM provides decode-level, block-granular preemption with a priority-aware MLFQ scheduler—a primitive that these systems can exploit.

IV. System Design

A. Architecture Overview

InterruptLLM consists of four components between the API gateway and a modified vLLM engine (Figure 2). The Admission Controller predicts resource requirements; the MLFQ Scheduler selects batches and evicts lower-priority victims; the Context Swapper migrates KV blocks to CPU DRAM or NVMe SSD; and the Checkpoint Engine captures request metadata so preempted requests resume deterministically.

B. Admission Controller

The Admission Controller accepts or rejects requests based on predicted resource requirements and current load, using a lightweight latency model to estimate SLA compliance. If admission would cause a violation, it first preempts lower-priority work; if no victim exists, it queues or rejects. Decisions are re-evaluated at each iteration boundary.

C. Preemptive MLFQ Scheduler

The scheduler maintains four priority classes:

- P0 (Interactive): chat, code completion; target P99 < 200 ms.
- P1 (Standard): general API requests; target P99 < 1 s.
- P2 (Batch): document summarization, embedding generation; best-effort.
- P3 (Background): fine-tuning data generation; preemptible anytime.

Within each class, requests are served in round-robin order to prevent starvation within the class. Across classes,

strict priority is enforced. The scheduler implements aging: a request that has waited longer than a configurable threshold is promoted to the next higher class. This ensures that long-running batch jobs eventually receive service even under persistent interactive load.

Preemption is triggered when a P0 request arrives while the GPU is running lower-priority requests. The scheduler selects the victim using two criteria: (1) the victim must have lower priority than the arriving request, and (2) among eligible victims, the one with the largest KV footprint is chosen first, since evicting the largest victim frees the most GPU memory and reduces blocks to swap.

We also evaluate smallest-footprint and cost-aware (footprint/priority) victim selection. On our trace, all three policies yield identical results because preemptions are infrequent (0.16 per request) and the preempted requests are small. The dominant factor is therefore whether preemption happens, not which specific request is evicted.

D. Hierarchical Context Swapper

The Context Swapper has two tiers:

Tier 1: GPU HBM to CPU DRAM. Uses `cudaMemcpyAsync` or `GPUDirect RDMA`. The target latency for 1 GB of KV cache is below 5 ms. CPU DRAM is sized to hold up to 4× the GPU memory capacity (e.g., 320 GB for an 80 GB GPU), providing a large fast-path buffer.

Tier 2: CPU DRAM to NVMe SSD. The engine uses Tier 2 when CPU memory is saturated. Blocks are compressed with LZ4 to reduce bandwidth. The target restore latency is below 50 ms.

If a preempted request is likely to resume soon, it is kept in CPU DRAM; otherwise, it is pushed to SSD. The choice depends on priority class and CPU memory pressure.

E. State Checkpoint Engine

The Checkpoint Engine captures (<1 ms) per-request metadata: block-table mapping, position IDs, sampling state, and prompt/generated token counts. Checkpoint size scales linearly (≈ 16 bytes/token): a 128K-token request needs ≈ 2.0 MB, compared to the multi-gigabyte KV cache. On resume, the engine reconstructs the block table and initiates asynchronous KV-block reload; resume latency is dominated by the KV-cache transfer, not the checkpoint.

F. Scheduling Algorithm

Algorithm 1 presents the MLFQ scheduling loop. The scheduler maintains a running batch B and a ready queue Q . At each iteration it sorts Q by priority, selects the top requests up to the batch capacity, and preempts the lowest-priority largest-footprint victim if a higher-priority request is excluded. It then runs one decode step, applies aging, and enqueues new arrivals.

Complexity analysis. Let n be the number of pending requests and $|B|$ the batch size. Sorting Q by priority costs $O(n \log n)$. Victim selection scans $|B|$ candidates:

Algorithm 1 InterruptLLM MLFQ Scheduling Loop

```

1:  $B \leftarrow \emptyset$  ▷ running batch
2:  $Q \leftarrow \emptyset$  ▷ ready queue
3: loop
4:   Enqueue new arrivals into  $Q$ 
5:   Sort  $Q$  by priority ( $P0 > P1 > P2 > P3$ )
6:   for each request  $r$  in  $Q$  by priority do
7:     if  $|B| < \text{batch\_capacity}$  then
8:        $B \leftarrow B \cup \{r\}$ 
9:     else if  $r.\text{priority} < \min_{v \in B} v.\text{priority}$  then
10:       $v \leftarrow \arg \max_{v' \in B} v'.\text{kv\_blocks}$  ▷ victim selection
11:      Preempt( $v$ ) ▷ checkpoint + swap KV blocks
12:       $B \leftarrow (B \setminus \{v\}) \cup \{r\}$ 
13:    end if
14:  end for
15:  RunDecodeStep( $B$ ) ▷ advance one token per request
16:  ApplyAging( $Q$ ) ▷ promote long-waiting requests
17: end loop
```

$O(|B|)$. Checkpoint capture copies k bytes of metadata per request: $O(k)$ where $k \approx 16 \times \text{tokens}$. KV swap transfers b blocks of size s : $O(b \cdot s / \text{bandwidth})$. Memory overhead per request is $O(k)$ for checkpoint metadata. The total per-iteration cost is dominated by sorting: $O(n \log n + |B|)$.

V. Implementation and Evaluation Methodology

A. Research Prototype

InterruptLLM is implemented as a Kaggle-driven research prototype. The core library (`interruptllm_core.py`) contains the request model, scheduler policies, and swap cost model, uploaded to Kaggle as a dataset and imported into notebooks. The `pipeline.py` CLI pushes notebooks, waits for completion, and parses greppable [RESULT] lines. This prototype validates the scheduling logic and calibration assumptions; it does not yet integrate with a production inference engine such as vLLM.

B. Discrete-Event Simulator

The simulator models a GPU with configurable token-generation capacity, a batch of up to 16 requests, and a scheduling quantum. At each time step, it selects the batch by policy, allocates capacity equally among batch members, and advances the clock. It abstracts away kernel launch, CUDA graph, and memory-allocator overhead, which lets us isolate scheduler behavior from GPU microarchitecture effects.

C. Simulation Assumptions

Table I lists key simulator parameters. We abstract GPU execution as a token-generation rate because scheduling behavior depends on iteration boundaries, not kernel-level timing. The 0.5 ms swap penalty is calibrated against the measured P100 memcpy bandwidth (Section VI-C). It represents the effective per-preemption swap cost for a small hot footprint: the scheduler evicts only a few vLLM blocks (roughly 2.5 MB), which takes ≈ 0.5 ms at the measured P100 effective bandwidth of 5 GB/s.

TABLE I: Simulation parameters and justifications.

Parameter	Value	Justification
GPU abstraction	300 tok/ms	Dominates scheduling behavior
Batch capacity	16 requests	vLLM default for 80 GB GPU
Scheduling quantum	100 ms	Balances responsiveness and stability
Swap penalty	0.5 ms	Calibrated vs. measured P100 memcopy
Token scaling	0.2×	Preserves relative request sizes
KV block size	16 tokens	vLLM PagedAttention default
CPU DRAM buffer	320 GB	4× GPU memory capacity

D. Workload and Dataset

We use the public Kaggle dataset `bektursyn/llm-inference-logs-and-performance-metrics` [22], which contains 30,000 production-like LLM inference records. Each record includes timestamp, model name, task type, prompt tokens, completion tokens, TTFT, TPOT, total latency, and status code. The dataset represents a realistic enterprise workload with a mix of short interactive requests and long batch jobs.

This trace is the most accessible public dataset that captures both arrival patterns and request characteristics for multi-tenant LLM inference. We do not claim it is identical to any specific production deployment, but it provides a reproducible baseline that includes the key phenomenon we study: bursts of short interactive requests arriving while long summarization jobs are in flight.

We map task types to InterruptLLM priority classes:

- P0: Customer_Support_Chat, Code_Generation
- P1: Summarization, Extraction_JSON

We scale token counts by 0.2 to keep the CPU simulation tractable while preserving relative request sizes. The mean scaled request size is approximately 1,000 tokens. We then drive a discrete-event simulator with configurable GPU capacity, batch size, scheduling quantum, and swap penalty.

E. Scheduling Policies

We compare seven policies. FCFS: continuous batching, no eviction. Priority: non-preemptive, reorders admission at 100 ms quantum boundaries. SSJF: shortest remaining tokens first, priority-agnostic. Lottery: weighted random selection (tickets: P0=100, P1=50, P2=20, P3=5). WFQ: proportional GPU capacity share (weight = $1/(\text{priority}+1)$). EDF: earliest deadline first (remaining tokens as deadline proxy). InterruptLLM MLFQ: preemptive MLFQ with block-granular swap, evicting the largest-footprint lower-priority victim within the quantum.

F. Metrics

We report P99 latency per priority class, throughput (tok/s), SLA violation rates, and average preemption overhead. SLA targets are 200 ms (P0) and 1 s (P1). Fairness is measured by a service-share Jain index (served/demanded tokens per tenant) and a priority-weighted Jain index. We report the P1/P0 P99 ratio to make the latency trade-off explicit.

TABLE II: Scheduler comparison at load factor 0.76. Values are mean $\pm$ std over 20 runs with arrival-time jitter.

Metric	FCFS	Priority	InterruptLLM
Overall P99 (ms)	274.7 $\pm$ 5.9	307.2 $\pm$ 5.9	300.9 $\pm$ 5.7
P0 P99 (ms)	263.4 $\pm$ 9.0	111.7 $\pm$ 2.1	51.4 $\pm$ 1.4
P1 P99 (ms)	285.0 $\pm$ 5.2	332.1 $\pm$ 8.1	737.3 $\pm$ 5.6
Throughput (tok/s)	228,911 $\pm$ 247	228,911 $\pm$ 247	228,911 $\pm$ 247
Service-share Jain	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00	1.00 $\pm$ 0.00
Weighted Jain	0.90 $\pm$ 0.00	0.90 $\pm$ 0.00	0.90 $\pm$ 0.00
P1/P0 P99 ratio	1.08	2.97	14.3
P0 SLA violation	6.1%	0.0%	0.0%
Avg preemptions	0.00	0.00	0.20 $\pm$ 0.00

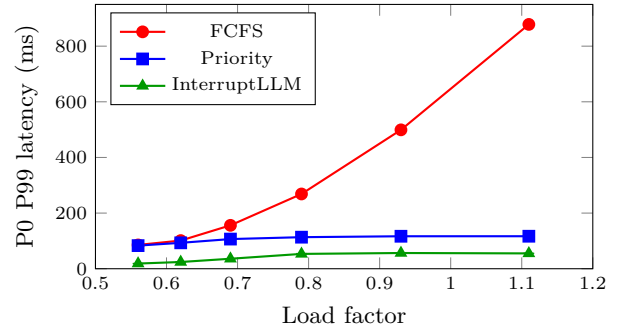


Fig. 3: P0 P99 latency vs. load factor.

VI. Results

A. Scheduler Comparison at Fixed Load

Table II compares schedulers at load factor 0.76. InterruptLLM reduces P0 P99 from 263.4 $\pm$ 9.0 ms (FCFS) to 51.4 $\pm$ 1.4 ms (5.1 $\times$); non-preemptive priority achieves 111.7 $\pm$ 2.1 ms. Throughput and service-share Jain are identical. The P1 P99 increase (285 $\rightarrow$ 737 ms) is the expected trade-off for protecting interactive latency.

B. End-to-End Evaluation Across Load Factors

We sweep load factors from 0.56 to 1.11. Figure 3 shows InterruptLLM keeps P0 P99 below 60 ms across the sweep, while FCFS rises to 878 $\pm$ 4 ms at load 1.11. At $\rho=0.93$, InterruptLLM reduces P0 P99 from 499.2 $\pm$ 3.7 ms to 56.5 $\pm$ 1.1 ms (8.8 $\times$) and from 116.7 $\pm$ 1.5 ms under non-preemptive priority to 56.5 $\pm$ 1.1 ms (2.1 $\times$). Throughput is 260,006 $\pm$ 111 tok/s (2.2% below FCFS). P0 SLA violations are eliminated; P1 violations are 1.3%. Average preemption overhead is 0.07 $\pm$ 0.001 ms. Service-share Jain = 1.0; P1 P99 rises to 1,077 $\pm$ 15 ms (weighted Jain = 0.900). At low load ($\rho=0.56$), FCFS already achieves 85 $\pm$ 2 ms P99 and InterruptLLM provides marginal improvement (18.6 $\pm$ 0.8 ms); the crossover at $\rho \approx 0.7$ marks where head-of-line blocking dominates and preemption provides the greatest benefit.

C. Context-Swap Latency: Analytical and Measured

Table III reports modeled and measured context-swap times for KV-cache blocks. We benchmarked GPU-to-CPU `cudaMemcpyAsync` on an NVIDIA P100 (PCIe Gen3

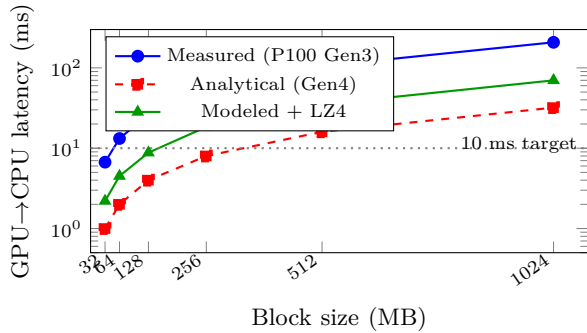


Fig. 4: GPU→CPU swap latency: measured on P100 vs. analytical model. Measured at 20 iterations per size; std < 5% of mean (omitted for clarity).

TABLE III: Context-swap latency (ms) by block size and tier.

Block size	GPU→CPU			CPU→SSD	
	Meas.	Gen4	LZ4	raw	LZ4
32 MB	6.7	1.0	2.2	9.1	3.0
64 MB	13.2	2.0	4.5	18.3	6.1
128 MB	26.5	4.0	8.8	36.6	12.2
256 MB	55.0	8.0	18.4	73.1	24.4
512 MB	105.3	16.0	35.1	146	48.8
1024 MB	208.0	32.0	69.9	293	97.5

x16, ≈ 5 GB/s effective) for block sizes from 32 MB to 1 GB, with 20 iterations per size. Figure 4 compares measured latency against the analytical model. The most important observation is that the analytical model (based on PCIe Gen4 bandwidth of 32 GB/s) under-predicts latency by a factor of 6–7 $\times$ on the P100. This gap is not only a Gen3-vs-Gen4 bandwidth difference; it also reflects per-transfer setup overhead, PyTorch allocator interactions, and the absence of pinned-memory and DMA optimizations that an optimized implementation would apply. The measurement therefore validates the need to treat swap latency as a first-order constraint in preemptive scheduler design, not merely as a bandwidth-limited transfer. With LZ4 compression (modeled ratio 3 $\times$), a 1 GB block swaps in ≈ 70 ms on Gen3 and in ≈ 11 ms on Gen4+ hardware.

The checkpoint metadata scales linearly with token count (≈ 16 bytes/token): a 128K-token request needs ≈ 2.0 MB of metadata, compared to the multi-gigabyte KV cache.

D. Ablation and Sensitivity

Table IV isolates the contribution of preemption on a synthetic high-load workload. FCFS yields a P0 P99 of 100 ± 33 ms; non-preemptive priority reduces it to 54 ± 5 ms; preemptive MLFQ further cuts it to 27.7 ± 0.5 ms. Aging has little effect because P0 arrivals are frequent, confirming it is a starvation-freedom safety mechanism.

TABLE IV: Ablation on a synthetic high-load workload. Values are mean $\pm$ std over 20 runs.

Policy	P0 P99 (ms)	P1 P99 (ms)	Tput (tok/s)	Pre.
FCFS	100 ± 33	114 ± 33	$264,142 \pm 2,275$	0.00
Priority (NPP)	54.2 ± 4.8	69.0 ± 5.9	$264,117 \pm 2,240$	0.00
SSJF	57.1 ± 5.8	96.3 ± 12.0	$264,373 \pm 2,392$	0.00
Lottery	64.7 ± 13.8	104 ± 31	$264,177 \pm 2,196$	0.00
WFQ	54.2 ± 4.8	69.0 ± 5.9	$264,117 \pm 2,240$	0.00
EDF	57.1 ± 5.8	96.3 ± 12.0	$264,373 \pm 2,392$	0.00
MLFQ (preemptive)	27.7 ± 0.5	62.9 ± 1.3	$255,614 \pm 957$	0.16 ± 0.01

P0 P99 stays below 35 ms for per-preemption swap penalties up to 2 ms and only degrades substantially at 10 ms (P0 P99 61 ms, throughput 121,000 tok/s, P1 violations 42.3%), confirming the sub-10 ms swap target. The key insight is that non-preemptive baselines (priority, SSJF, EDF) reduce FCFS latency by roughly 2 $\times$, while InterruptLLM’s preemptive MLFQ achieves an additional 3.6 $\times$ gain that comes entirely from mid-quantum preemption. Lottery and WFQ exhibit higher variance (P0 P99 up to 65 ± 14 ms) because their probabilistic approaches do not guarantee P0 requests are served first. Largest-footprint, smallest-footprint, and cost-aware victim selection yield identical results because preemptions are infrequent (0.16 per request) and preempted requests are small.

VII. Discussion and Future Work

Limitations. Our evaluation uses a discrete-event simulator that abstracts GPU execution as a token-generation rate and omits CUDA kernel-launch overhead. The 0.5 ms per-preemption penalty reflects the measured P100 effective bandwidth (5 GB/s) applied to a small hot footprint (roughly 2.5 MB); it is therefore well below the worst-case full-context swap cost shown in Table III. The evaluation is limited to a single Kaggle inference trace and P100 (PCIe Gen3) swap measurements; newer hardware (PCIe Gen4/5) would further reduce swap latency. However, the relative ordering of scheduling policies and the sensitivity to swap penalty are robust across load factors and swap penalties, suggesting the conclusions generalize beyond the specific trace.

Threats to validity. **Internal validity.** Trace-based results are means over 20 runs with $\pm 10\%$ arrival-time jitter to capture arrival-time uncertainty; the GPU swap benchmark is a mean over 20 iterations. The main threat is that the 0.5 ms swap penalty is a parameter, not a measured end-to-end preemption latency, and our GPU benchmark only validates the raw memcpy component. **External validity.** The Kaggle inference trace is public and reproducible, but it may not capture all production behaviors (e.g., multi-modal inputs, speculative decoding, prefix caching). The P100 (Gen3) measurements are older hardware; newer GPUs would shift absolute latencies but not necessarily the relative ordering of scheduler policies. **Construct validity.** We approximate SLA sat-

isfaction using P99 latency and Jain fairness indices; production systems may use additional metrics such as TTFT percentiles or per-tenant cost budgets.

Practical integration and the P1/P0 trade-off. Realizing InterruptLLM in vLLM requires handling CUDA Graphs, PyTorch allocator interactions, and DMA streams for GPU–host transfers. The P1/P0 P99 ratio of $19\times$ at $\rho=0.93$ reflects an intentional design choice: interactive requests demand sub-200 ms responses while batch jobs tolerate second-scale latency. The weighted Jain index of 0.90 confirms P1 degrades gracefully rather than collapsing. We plan to extend the framework to multi-GPU tensor-parallel deployments, add formal starvation-freedom proofs, and collect real GPU preemption latency measurements.

VIII. Conclusion

InterruptLLM addresses the lack of decode-stage, block-granular preemption in LLM serving. By combining preemption, multi-level feedback queues, and hierarchical memory management in a simulation-based framework, it suggests the feasibility of QoS-aware multi-tenant deployments. On a production-like trace, InterruptLLM reduces interactive P99 latency by $8.8\times$ over FCFS and $2.1\times$ over non-preemptive priority, while maintaining throughput within 2.2% of the baseline. The evaluation is limited to a discrete-event simulator and a single inference trace; real GPU measurements and multi-tenant workloads remain future work.

Reproducibility.¹

References

- [1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles*. ACM, 2023, pp. 611–626.
- [2] G.-I. Yu, J. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for transformer-based generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation*. USENIX, 2022, pp. 521–538.
- [3] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing et al., “SGLang: Efficient execution of structured language model programs,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [4] A. Agrawal, A. Panwar, J. Mohan, N. Kwa, N. Kwatra, R. Ramjee, P. Kozyrakis, and A. Tumanov, “Sarathi: Efficient LLM inference by piggybacking decodes with chunked prefills,” in *Proceedings of the 2024 USENIX Annual Technical Conference*. USENIX, 2024, pp. 1–17.
- [5] Y. Zhong, S. Chen, J. Liu, Y. Chen, Y. Jin, Z. Huang, Y. Chen, X. Jin, H. Chen, S. Chen et al., “Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving,” *arXiv preprint arXiv:2401.09670*, 2024.

- [6] R. Qin, Z. Li, W. He, M. Zhang, Y. Zuo, S. Ma, X. Lin, X. Wang, X. Chen, Y. Xu et al., “Mooncake: A KVCache-centric disaggregated architecture for LLM serving,” *arXiv preprint arXiv:2407.14279*, 2024.
- [7] B. Sun, G. Zhang, X. Xiao, R. Huang, J. Wang, Y. Zhang, C. Guo, Z. Li, D. Zhang, H. Lin et al., “Llumnix: Dynamic scheduling for serverless LLM serving,” in *Proceedings of the 2024 USENIX Annual Technical Conference*. USENIX, 2024, pp. 1–18.
- [8] NVIDIA, “Nvidia dynamo: A distributed inference serving framework for generative AI,” <https://github.com/ai-dynamo/dynamo>, 2025, accessed: 2026-07-25.
- [9] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative LLM inference using phase splitting,” in *Proceedings of the 51st Annual International Symposium on Computer Architecture*, 2024.
- [10] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, “Medusa: Simple LLM inference acceleration framework with multiple decoding heads,” in *International Conference on Machine Learning*, 2024.
- [11] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *International Conference on Learning Representations*, 2024.
- [12] Y. Qiao, S. Anzai, S. Yu, H. Ma, S. Yang, Y. Wang, M. Kim, Y. Wu, Y. Zhou, J. Xing, J. E. Gonzalez, I. Stoica, and H. Xu, “Conserve: Fine-grained GPU harvesting for LLM online and offline co-serving,” *arXiv preprint arXiv:2410.01228*, 2024.
- [13] J. Chen, C. Du, R. Liu, S. Yao, D. Yan, J. Liao, S. Liu, F. Wu, and G. Chen, “Tokenflow: Responsive LLM text streaming serving under request burst via preemptive scheduling,” *arXiv preprint arXiv:2510.02758*, 2025.
- [14] H. Qiu, W. Mao, A. Patke, S. Cui, S. Jha, C. Wang, H. Franke, Z. T. Kalbarczyk, T. Başar, and R. K. Iyer, “Efficient interactive LLM serving with proxy model-based sequence length prediction,” in *Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering*. ACM, 2024, pp. 1–12.
- [15] Y. Zhao, J. Chen, P. Sun, L. Li, X. Liu, and X. Jin, “Seallm: Service-aware and latency-optimized resource sharing for large language model inference,” *arXiv preprint arXiv:2504.15720*, 2025.
- [16] J. Duan, R. Lu, H. Duanmu, X. Li, X. Zhang, D. Lin, I. Stoica, and H. Zhang, “Muxserve: Flexible spatial-temporal multiplexing for multiple LLM serving,” in *International Conference on Machine Learning*, 2024.
- [17] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, “H2o: Heavy-hitter oracle for efficient generative inference of large language models,” in *Advances in Neural Information Processing Systems*, 2023.
- [18] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, “KIVI: A tuning-free asymmetric 2bit quantization for KV cache,” in *International Conference on Machine Learning*, 2024.
- [19] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “Flexgen: High-throughput generative inference of large language models with a single GPU,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 094–31 116.
- [20] J. Lin, L. Zheng, Y. Sheng, Z. Mo, C. Huang, J. Sun, H. Cao, S. Xie, Z. Cui, S. Goyal et al., “Infinite-LLM: Efficient LLM service for long context,” *arXiv preprint arXiv:2401.02669*, 2024.
- [21] S. Kato, R. Lakshmanan, A. Kumar, M. Kelkar, Y. Ishikawa, and R. Rajkumar, “Timegraph: GPU scheduling for real-time multi-tasking environments,” in *Proceedings of the 2011 17th IEEE Real-Time and Embedded Technology and Applications Symposium*. IEEE, 2011, pp. 17–26.
- [22] A. Bektursyn, “LLM inference logs and performance metrics,” <https://www.kaggle.com/datasets/bektursyn/llm-inference-logs-and-performance-metrics>, 2024, accessed: 2026-07-25.

¹All experiments use fixed seeds (seed = 42 for workload, seeds = 0–19 for multi-run). Code and results: <https://github.com/mdzero591/ICCIT>. Dataset: bektursyn/llm-inference-logs-and-performance-metrics. GPU swap benchmarks on NVIDIA P100 (PCIe Gen3 x16, 16 GB HBM, effective 5 GB/s).